

Project #3 guidelines for Python

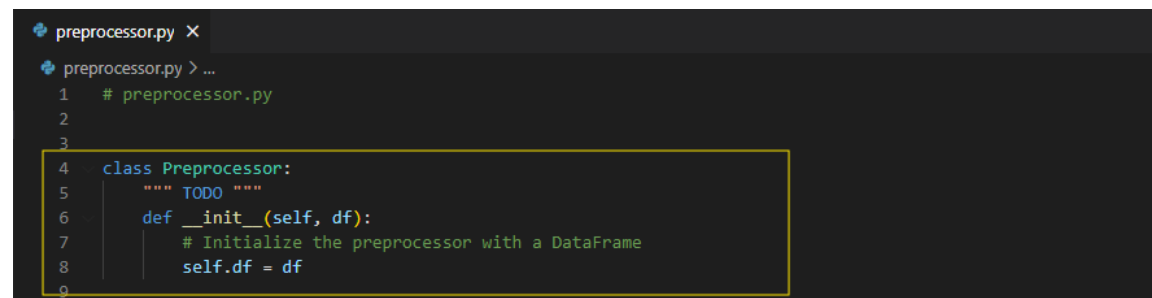
If you decide to write your project using python, I'll tell you where to implement the code in project #3 framework.

Implementation

a. Use your custom data preprocessing methods

- a. Complete ***Preprocessor class*** in *preprocessor.py*

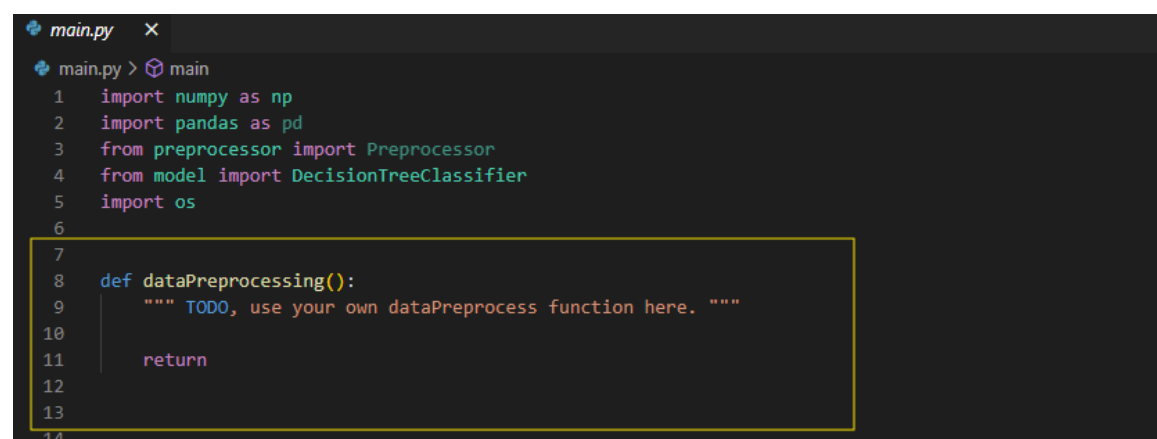
Use your custom data preprocessing functions in project #1 as the methods in Preprocessor class.



```
preprocessor.py X
preprocessor.py > ...
1 # preprocessor.py
2
3
4 class Preprocessor:
5     """ TODO """
6     def __init__(self, df):
7         # Initialize the preprocessor with a DataFrame
8         self.df = df
9
```

- b. Complete ***DataPreprocessing function*** in *main.py*

Use the custom methods in project #1 you implemented to complete dataPreprocessing function.



```
main.py X
main.py > main
1 import numpy as np
2 import pandas as pd
3 from preprocessor import Preprocessor
4 from model import DecisionTreeClassifier
5 import os
6
7
8 def dataPreprocessing():
9     """ TODO, use your own dataPreprocess function here. """
10
11     return
12
13
14
```

Note: You can either use your data preprocessing methods in project #1 or implement a new one for project #3.

b. Complete Decision Tree in *model.py*

- Implement necessary methods in *DecisionTreeClassifier*
(You don't have to follow the template to implement your decision tree)

```
class DecisionTreeClassifier:
    def __init__(self, max_depth=1):
        self.max_depth = max_depth

    def fit(self, X, y):
        self.tree = self._grow_tree(X, y)

    def _grow_tree(self, X, y, depth=0):
        raise NotImplementedError

    # Split dataset based on a feature and threshold
    def split_dataset(X, y, feature_index, threshold):
        raise NotImplementedError

    # Find the best split for the dataset
    def find_best_split(X, y):
        raise NotImplementedError

    def entropy(y):
        raise NotImplementedError

    # prediction
    def predict_proba(self, X):
        raise NotImplementedError

    def predict(self, X):
        raise NotImplementedError

    def _predict_tree(self, x, tree_node):
        raise NotImplementedError

    # print tree
    def print_tree(self, max_print_depth=3):
        raise NotImplementedError
```

c. Complete main function in *main.py*

- Build your decision tree and save the predict label as .csv file

```
def main():  
    root_path = r"yourtraining_data" # change the root path  
    train_X = os.path.join(root_path, "train_x.csv")  
    train_y = os.path.join(root_path, "train_y.csv")  
    test_X = os.path.join(root_path, "test_x.csv")
```

```
    Decision_tree = DecisionTreeClassifier(max_depth=1)  
    Decision_tree.fit(train_X,train_y)
```

```
    # TODO  
    # build your decision tree  
    # predict the output of the testing data  
    # remember to save the predict label as .csv file
```

```
if __name__ == "__main__":  
    np.random.seed(0)  
    main()
```